

Universidad San Carlos de Guatemala

Centro universitario de Occidente



División de Ciencias de la ingeniería

Ingeniería en Ciencias y Sistemas

Estructura De Datos

Ing. Mario Ramirez Tobar

REPORT

PROYECTO 1 FASE 1

GESTIÓN DE CATALOGO DE PRODUCTOS DE SUPERMERCADO

Cristian Alejandro Roldán López

Carné: 202147280

Quetzaltenango 03 de abril de 2026

1. Introducción

El presente proyecto implementa un sistema de gestión de inventario para un supermercado utilizando cuatro estructuras de datos complementarias en C++17. Cada estructura se seleccionó con base en la operación que optimiza: el Árbol AVL para búsqueda exacta por nombre en $O(\log n)$, el Árbol B para consultas de rango por fecha de caducidad, el Árbol B+ para listado eficiente por categoría gracias a sus hojas enlazadas, y la Lista Enlazada como estructura base para comparativas de rendimiento mediante búsqueda secuencial.

Los productos se cargan desde archivos CSV con validación robusta de campos y registro de errores, y el sistema expone un menú interactivo con colores ANSI, preview de árboles en consola y exportación a formato DOT para visualización con Graphviz.

2. Diseño de las Estructuras

2.1 Clase Producto

La clase Producto es el tipo de dato fundamental del sistema. Almacena siete atributos con acceso mediante getters/setters:

Atributo	Tipo	Descripción
nombre	string	Nombre comercial del producto
codigo	string	Código de barras (clave única)
categoria	string	Categoría (clave del Árbol B+)
fechaCaducidad	string	Fecha YYYY-MM-DD (clave del Árbol B)
marca	string	Marca del fabricante
precio	double	Precio de venta (Q)
cantidad	int	Stock disponible

2.2 Lista Enlazada Simple

Implementación de lista enlazada sin orden compuesta por nodos (Nodo) que almacenan un objeto Producto y un puntero al siguiente. Sirve como

estructura base para comparativas de búsqueda secuencial frente al Árbol AVL.

Operación	Descripción	Complejidad
insert(p)	Inserta al final de la lista	O(n)
imprimir()	Recorre e imprime todos los nodos	O(n)
buscar(nombre)	Búsqueda secuencial por nombre	O(n)
eliminar(codigo)	Elimina por código de barras	O(n)
remove(p)	Elimina por objeto Producto	O(n)

Nota: Para la versión ordenada, la inserción busca la posición correcta (O(n)) y la búsqueda puede detenerse cuando la clave supera el valor buscado, reduciendo el caso promedio a O(n/2), aunque el peor caso permanece O(n).

2.3 Árbol AVL — Indexado por Nombre

Árbol binario de búsqueda autobalanceado. Cada nodo (NodoAvl) almacena un objeto Producto, punteros izquierdo/derecho y su altura. El balance se mantiene con rotaciones simples y dobles después de cada inserción o eliminación.

Propiedades del nodo

- Clave: nombre del producto (comparación lexicográfica)
- Factor de balance: altura(izq) - altura(der) en {-1, 0, 1}
- Altura garantizada: $h \leq 1.44 * \log(n+2)$

Rotaciones implementadas

Caso	Condición	Acción
LL (Simple Der.)	balance > 1, nuevo < raiz.izq	rotacionDerecha(y)
RR (Simple Izq.)	balance < -1, nuevo > raiz.der	rotacionIzquierda(x)

LR (Doble)	balance > 1, nuevo > raiz.izq	rotlzq(izq) + rotDer
RL (Doble)	balance < -1, nuevo < raiz.der	rotDer(der) + rotlzq

2.4 Árbol B — Indexado por Fecha de Caducidad

Árbol B con grado mínimo $t=3$ (máximo 5 claves por nodo). La clave es la fecha de caducidad convertida a entero YYYYMMDD para comparación eficiente. Permite búsqueda exacta y por rango.

Parámetros del árbol ($t = 3$)

Parámetro	Valor	Significado
Grado mínimo (t)	3	Nodo no-raíz tiene al menos $t-1 = 2$ claves
Máx. claves/nodo	$2t-1 = 5$	Nodo lleno tiene 5 claves
Máx. hijos/nodo	$2t = 6$	Nodo lleno tiene 6 punteros hijo
Altura máxima	$\log_t((n+1)/2)$	Garantía logarítmica para n elementos

La conversión a entero (fechaAEntero) permite comparar fechas con operadores aritméticos enteros, evitando comparaciones de cadenas y acelerando las consultas de rango. La búsqueda de rango recorre el árbol en in-order acotando la exploración con $O(\log n + k)$.

2.5 Árbol B+ — Indexado por Categoría

Árbol B+ con grado mínimo $t=3$. A diferencia del Árbol B, todos los datos se almacenan en las hojas, y las hojas están doblemente enlazadas, lo que permite recorrer todos los productos de una categoría sin volver a subir al árbol una vez encontrada la primera hoja.

- Nodos internos: almacenan solo claves de navegación (categoría)
- Nodos hoja: almacenan clave + objeto Producto completo
- Lista enlazada de hojas: punteros siguiente/anterior para recorrido $O(k)$
- División de hojas: la clave mediana se COPIA al padre (no se mueve)

- Division de internos: la clave mediana SUBE al padre y se elimina del hijo

3. Complejidad Big-O y Justificación

La siguiente tabla resume la complejidad temporal de todas las operaciones del sistema. n = número de productos; k = resultados que coinciden con el criterio; h = altura del árbol.

Estructura	Operación	Complejidad	Justificación
Lista Enlazada (desordenada)	Inserción al final	$O(n)$	Recorre hasta el último nodo
	Búsqueda secuencial	$O(n)$	Peor caso: recorre todos los nodos
	Eliminación	$O(n)$	Búsqueda previa $O(n)$ + reenlace $O(1)$
Lista Enlazada (ordenada)	Inserción ordenada	$O(n)$	Busca posición de inserción linealmente
	Búsqueda	$O(n/2)$ prom.	Puede detenerse si clave > buscada
	Eliminación	$O(n)$	Búsqueda $O(n)$ + reenlace $O(1)$
Árbol AVL (por nombre)	Inserción	$O(\log n)$	Altura $h \leq 1.44 \log n$; rotaciones $O(1)$
	Búsqueda exacta	$O(\log n)$	Un nivel por comparación en árbol balanceado
	Eliminación	$O(\log n)$	Busca $O(\log n)$ + sucesor + rebalanceo
	Recorrido in-order	$O(n)$	Visita los n nodos exactamente una vez
Árbol B (por fecha)	Inserción	$O(\log n)$	Altura $O(\log_t n)$, división sube en $O(1)/\text{nivel}$
	Búsqueda exacta	$O(\log n)$	Busca entre $O(t)$ claves/nodo, h niveles

	Búsqueda rango	$O(\log n + k)$	Localiza inicio $O(\log n)$, imprime k resultados
	Eliminación	$O(\log n)$	Localizar + préstamo/fusión si necesario
Árbol B+ (por categoría)	Inserción	$O(\log n)$	Copia clave mediana en hojas (no mueve)
	Búsqueda exacta	$O(\log n)$	Datos en hojas; baja por internos
	Búsqueda por categoría	$O(\log n + k)$	Localiza hoja + sigue lista enlazada
	Recorrido hojas	$O(n)$	Sigue lista enlazada de extremo a extremo
	Eliminación	$O(\log n)$	Busca en hoja + préstamo/fusión

Nota sobre la Lista Enlazada Ordenada

La lista ordenada mejora el caso promedio de búsqueda a $O(n/2)$ al abortar cuando se supera la clave buscada, pero el peor caso sigue siendo $O(n)$. Comparada con el AVL, es inferior en todos los casos asintóticos; sin embargo, tiene constantes más bajas para n pequeño (< 50 elementos) por la ausencia de rotaciones.

4. Metodología de Medición

4.1 Herramienta de Cronometría

Todas las mediciones se realizan con `std::chrono::high_resolution_clock` de la biblioteca estándar C++17. Se captura el timestamp en microsegundos (μs) antes y después de cada operación:

```
long long t0 = duration_cast<microseconds>(
    high_resolution_clock::now().time_since_epoch()).count();
long long dt = duration_cast<microseconds>(
    high_resolution_clock::now().time_since_epoch()).count() - t0;
```

4.2 Opción 12 — Comparar Tiempos

El menú expone la opción 12 — Comparar tiempos de búsqueda. Dado un nombre de producto, ejecuta la búsqueda en el AVL y en la lista enlazada e imprime ambos tiempos con una clasificación visual: ultra-rápido ($< 10 \mu\text{s}$), rápido ($< 100 \mu\text{s}$), normal ($< 1 \text{ ms}$), lento ($\geq 1 \text{ ms}$).

4.3 Resultados Estimados (Teóricos)

La siguiente tabla muestra tiempos estimados para distintos tamaños de catálogo basados en las complejidades asintóticas. Los valores son orientativos; las mediciones reales varían según hardware, caché y distribución de datos.

n (productos)	AVL buscar $O(\log n)$	Lista buscar $O(n)$	Factor aprox.
100	$\sim 0.5 \mu\text{s}$	$\sim 5 \mu\text{s}$	$\sim 10\text{x}$
1,000	$\sim 1.0 \mu\text{s}$	$\sim 50 \mu\text{s}$	$\sim 50\text{x}$

La ventaja del AVL crece exponencialmente con n . Para catálogos típicos de supermercado (1k–100k productos) la diferencia oscila entre 50x y 2,500x a favor del AVL.

5. Conclusiones

Complementariedad de estructuras: Ninguna estructura domina en todas las operaciones. El AVL es óptimo para búsqueda exacta por nombre; el Árbol B para rangos temporales; el B+ para agrupación por categoría; la lista enlazada para inserción simple cuando no se requiere búsqueda eficiente.

Árbol B+ y hojas enlazadas: La decisión de enlazar las hojas del Árbol B+ convierte el recorrido por categoría de $O(n \log n)$ a $O(\log n + k)$, una mejora crítica para catálogos con muchas categorías.

Árbol B y conversión a entero: Convertir la fecha a entero YYYYMMDD permite comparaciones enteras en lugar de lexicográficas, eliminando riesgos de error en formatos de fecha y mejorando la velocidad de comparación.

Lista Enlazada como referencia: La lista enlazada sin ordenar cumple su rol como estructura de referencia. Su $O(n)$ en búsqueda hace evidente la ventaja logarítmica del AVL, especialmente en catálogos grandes.

Inserción atómica: CargarCSV inserta cada producto en las cuatro estructuras con mecanismo de rollback, garantizando consistencia ante errores de validación (campos vacíos, precio no numérico, stock no entero).

Escalabilidad: La arquitectura es escalable: agregar un nuevo índice requiere solo añadir las llamadas en insertarEnTodo y eliminarDeTodo de main.cpp, sin modificar las estructuras existentes.